

CITADEL

Summer School · AI Track 2026

Fiches Volontaires

Phase 1 - Déployer un classificateur de sentiment

#	Volontaire	Horaire	Mission principale
V1	E1-E2 — Training	8h30 – 9h15	<i>01_train.py</i> : préparer données, TF-IDF, LogReg, évaluer, sauvegarder
V2	E3-E4 — API	9h15 – 10h00	<i>02_serve.py</i> : charger modèle, <i>predire()</i> , endpoint <i>/predict</i>
V3	E5 — Interface	10h00 – 10h45	<i>02_serve.py</i> : <i>/health</i> , <i>/app</i> + expliquer front ↔ back
V4	E6 — Docker	10h45 – 11h30	<i>Dockerfile</i> : builder l'image, monter le volume modèle
V5	E7 — Tests (1/2)	11h30 – 12h00	<i>03_test.py</i> : health check + prédictions (TODO 1 & 2)
V6	E7 — Tests (2/2)	12h00 – 12h15	<i>03_test.py</i> : cas limite texte vide (TODO 3)
V7	Débrief	12h15 – 13h00	Synthèse, questions de réflexion, perspectives



Chaque volontaire *DEROULE* son bloc du TP devant la classe. Il explique ce qu'il fait, complète les *TODO* à voix haute, exécute le code. Les autres suivent sur leur machine et font de même.

V1

Training & Sérialisation

01_train.py - Compléter les 5 étapes du pipeline ML

🕒 8h30 – 9h15 · 45 min

🎯 TON RÔLE

Tu ouvres 01_train.py devant la classe, tu lis le problème à voix haute, tu complètes chaque TODO en expliquant ce que tu fais. La classe fait la même chose sur sa propre machine en même temps que toi.

📋 CE QUE TU DOIS EXPLIQUER AVANT DE CODER (5 MIN)

- Le problème : classer des avis clients en positif/négatif automatiquement
- Le pipeline ML : données → vectorisation → entraînement → évaluation → sauvegarde
- Pourquoi sauvegarder ? → Pour déployer sans réentraîner à chaque requête
- Ce que sont les artefacts : vectorizer.pkl, classifier.pkl, meta.json

📌 TODO À COMPLÉTER DANS L'ORDRE

#	Fonction à compléter	Ce qu'il faut faire	✓
1a	<code>preparer_donnees()</code>	textes = [t for t, _ in dataset] / labels = [l for _, l in dataset]	☐
2	<code>creer_vectoriseur()</code>	TfidfVectorizer(ngram_range=(1,2), max_features=5000)	☐
3a	<code>entraîner()</code>	vectoriseur.fit_transform(textes_train) → matrice TF-IDF	☐
3b	<code>entraîner()</code>	LogisticRegression(max_iter=1000).fit(X_train, labels_train) puis	☐
4a	<code>evaluer()</code>	vectoriseur.transform(textes_test) — ⚠ transform, pas fit_transform !	☐
4b	<code>evaluer()</code>	classifieur.predict(X_test)	☐
4c	<code>evaluer()</code>	print(classification_report(...))	☐
4d	<code>evaluer()</code>	accuracy_score(labels_test, predictions)	☐
5a	<code>sauvegarder()</code>	pickle.dump(vectoriseur) dans vectorizer.pkl	☐
5b	<code>sauvegarder()</code>	pickle.dump(classifieur) dans classifier.pkl	☐
5c	<code>sauvegarder()</code>	json.dumps({type, classes, accuracy}) dans meta.json	☐

► COMMANDE À EXÉCUTER

```
cd citadel_tp_etudiant/phase1_sentiment
python 01_train.py
```



Résultat attendu : Accuracy 83.3% · 3 fichiers créés dans model/sentiment/

? QUESTIONS À POSER À LA CLASSE

- « Pourquoi `.transform()` sur le test et `.fit_transform()` sur le train ? »
- « Que contient `vectorizer.pkl` ? Pouvez-vous l'ouvrir avec un éditeur texte ? »
- « Que se passe-t-il si on réentraîne le modèle — les `.pkl` sont-ils compatibles ? »



Erreur fréquente : utiliser `.fit_transform()` sur le test → le vocabulaire TF-IDF est recalculé, les scores sont faux. Bien insister sur cette distinction.

V2

API FastAPI — /predict

02_serve.py — Charger le modèle et exposer la prédiction



TON RÔLE

Tu reprends là où V1 s'est arrêté. Le modèle est sauvegardé dans `model/sentiment/`. Tu vas maintenant l'envelopper dans une API FastAPI pour qu'il soit accessible via HTTP.



CE QUE TU DOIS EXPLIQUER AVANT DE CODER (5 MIN)

- Pourquoi une API REST ? → N'importe qui peut interroger le modèle sans Python
- Le dictionnaire `state` : charger le modèle UNE fois au démarrage, pas à chaque requête
- Le `lifespan` FastAPI : code qui tourne au démarrage → charge le modèle en mémoire
- Le flux : `POST /predict` → `predire()` → résultat JSON



TODO À COMPLÉTER DANS L'ORDRE

#	Fonction à compléter	Ce qu'il faut faire	✓
1a	<code>charger_modele()</code>	<code>json.loads(meta_path.read_text())</code> → <code>meta</code>	☐
1b	<code>charger_modele()</code>	<code>pickle.load()</code> → <code>state['vectoriseur']</code>	☐
1c	<code>charger_modele()</code>	<code>pickle.load()</code> → <code>state['classifieur']</code>	☐
1d	<code>charger_modele()</code>	<code>state['meta'] = meta</code> / <code>state['classes'] = meta['classes']</code>	☐
2a	<code>predire()</code>	<code>vec.transform([texte])</code> — liste avec UN seul texte	☐
2b	<code>predire()</code>	<code>clf.predict(X)[0]</code> → <code>label_id</code> (0 ou 1)	☐
2c	<code>predire()</code>	<code>clf.predict_proba(X)</code> → <code>proba[0][label_id]</code>	☐
2d	<code>predire()</code>	<code>state['classes'][label_id]</code> → 'négatif' ou 'positif'	☐
3	<code>lifespan()</code>	Appeler <code>charger_modele()</code> avant le <code>yield</code>	☐
4a	<code>predire_sentiment()</code>	<code>if not corps.texte.strip(): raise HTTPException(400, ...)</code>	☐
4b	<code>predire_sentiment()</code>	<code>resultat = predire(corps.texte)</code>	☐
4c	<code>predire_sentiment()</code>	<code>return PredictionSortie(texte=..., modele=..., **resultat)</code>	☐

► COMMANDE À EXÉCUTER

```
uvicorn 02_serve:app --host 0.0.0.0 --port 8000 --reload
# Dans un second terminal :
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"texte": "Ce produit est excellent !"}'
```

✓ **Résultat attendu :** `{"label": "positif", "score": 0.94, "positif": true, "modele": "tfidf+logreg"}`

? QUESTIONS À POSER À LA CLASSE

- « Que se passerait-il si on chargeait le modèle dans `predire()` au lieu de `lifespan` ? »
- « `predict_proba()` retourne quoi exactement ? Affichez-le avec `print()` »
- « Pourquoi mettre `[texte]` (une liste) et non juste `texte` dans `transform()` ? »

⚠ **Erreur fréquente :** oublier d'appeler `charger_modele()` dans `lifespan` → `KeyError 'vectoriseur'` à la première requête POST.

V3

Interface & Health

02_serve.py — Endpoint `/health` et interface web `/app`

🕒 10h00 – 10h45 · 45 min

🎯 TON RÔLE

Tu complètes le dernier TODO de `02_serve.py (/health)`, puis tu montres l'interface web et tu expliques l'architecture front ↔ back à la classe.

📋 CE QUE TU DOIS EXPLIQUER AVANT DE CODER (5 MIN)

- `/health` : endpoint de monitoring — indispensable en production pour savoir quel modèle tourne
- `/app` sert le fichier `app/index.html` depuis FastAPI — pas de serveur web séparé
- L'interface HTML appelle POST `/predict` via `fetch()` en JavaScript
- Séparation des responsabilités : l'interface ne contient aucun code ML

📝 TODO À COMPLÉTER

#	Fonction à compléter	Ce qu'il faut faire	✓
5	<code>/health</code> → <code>etat()</code>	Retourner <code>{status, modele, accuracy, classes}</code> depuis <code>state</code>	☐

► VÉRIFICATION DE /HEALTH

```
curl http://localhost:8000/health
# → {"status": "ok", "modele": "tfidf+logreg", "accuracy": 0.833, ...}
```

```
# Dans le navigateur :
http://localhost:8000/app
# Tester les boutons exemples et saisir un texte libre
```



Montrer le code `fetch()` dans `app/index.html` : c'est du JavaScript pur qui appelle POST `/predict`. Pas de framework, pas de dépendance — n'importe quel navigateur peut l'exécuter.

? QUESTIONS À POSER À LA CLASSE

- « Pourquoi exposer `/health` ? Qui l'interroge en production ? »
- « Que se passe-t-il si `app/index.html` n'existe pas ? (tester le 404) »
- « Comment l'interface sait-elle quelle URL appeler ? (voir `API_BASE` dans le JS) »

V4 Docker

Containeriser le service pour un déploiement reproductible

🕒 10h45 – 11h30 · 45 min

🎯 TON RÔLE

Tu montres comment transformer tout ce qui a été construit (modèle + API + interface) en une image Docker portable, déployable sur n'importe quel serveur — y compris le cluster ANAM.

📋 CE QUE TU DOIS EXPLIQUER AVANT DE CODER (5 MIN)

- Problème sans Docker : 'ça marche chez moi' — Python 3.9 vs 3.11, lib manquante...
- Image Docker = recette complète avec Python 3.11 + toutes les dépendances
- Container = l'image qui tourne — isolé du système hôte
- Le volume (`-v`) : le modèle `.pkl` est monté depuis l'hôte → on peut réentraîner sans rebuild

💻 LIRE LE DOCKERFILE LIGNE PAR LIGNE

```
FROM python:3.11-slim          # image de base légère (~150 Mo)
WORKDIR /app                  # répertoire de travail dans le container
COPY requirements.txt .        # copier EN PREMIER → cache Docker préservé
RUN pip install -r requirements.txt
COPY phasel_sentiment/ ./phasel_sentiment/
WORKDIR /app/phasel_sentiment
CMD ["uvicorn", "02_serve:app", "--host", "0.0.0.0", "--port", "8000"]
```

► COMMANDES À EXÉCUTER

```
# Builder l'image (depuis citadel_tp_etudiant/)
docker build -f ../Dockerfile.phasel -t citadel-sentiment ..

# Lancer le container avec le modèle monté en volume
docker run -p 8000:8000 \
  -v ./phasel_sentiment/model:/app/phasel_sentiment/model \
  citadel-sentiment
```



Si Docker n'est pas installé : lire et commenter le Dockerfile ligne par ligne. La compréhension du concept est l'objectif, pas l'exécution obligatoire.

? QUESTIONS À POSER À LA CLASSE

- « Pourquoi copier requirements.txt avant le code ? (cache Docker) »
- « Que se passe-t-il si on réentraîne le modèle — faut-il rebuild l'image ? »
- « Comment déployer cette image sur 10 nœuds du cluster ANAM ? »

V5 Tests — Partie 1

03_test.py — Health check et prédictions (TODO 1 & 2)

🕒 11h30 – 12h00 · 30 min

🎯 TON RÔLE

Tu montres comment valider formellement qu'une API fonctionne — pas juste 'ça semble marcher' mais des tests automatisés qui passent ou échouent de façon reproductible.

📋 CE QUE TU DOIS EXPLIQUER AVANT DE CODER (5 MIN)

- Tests d'intégration : tester le système complet de bout en bout, pas juste les fonctions
- requests.get() / requests.post() : appels HTTP depuis Python
- reponse.status_code : 200 = OK, 400 = requête invalide, 500 = erreur serveur
- reponse.json() : décoder la réponse JSON en dictionnaire Python

📌 TODO À COMPLÉTER

#	Fonction à compléter	Ce qu'il faut faire	✓
1a	tester_health()	requests.get(f'{API_BASE}/health', timeout=5)	☐
1b	tester_health()	if reponse.status_code != 200: return False	☐
1c	tester_health()	data = reponse.json()	☐
1d	tester_health()	if data.get('status') != 'ok': return False	☐
2a	tester_prediction()	requests.post(f'{API_BASE}/predict', json={'texte': texte}, timeout=10)	☐
2b	tester_prediction()	data = reponse.json()	☐
2c	tester_prediction()	label_predit = data['label'] / score = data['score']	☐
2d	tester_prediction()	return label_predit == label_attendu	☐

► COMMANDE À EXÉCUTER (TERMINAL SÉPARÉ)

```
# L'API doit toujours tourner dans un autre terminal
python 03_test.py
# Tests 1 et 2 doivent passer – Test 3 est pour V6
```



Les 6 cas de prédiction sont fournis dans `CAS_DE_TEST`. V5 n'a pas à les écrire, seulement à implémenter la fonction `tester_prediction()` qui les parcourt.

? QUESTIONS À POSER À LA CLASSE

- « Quelle est la différence entre un test unitaire et un test d'intégration ? »
- « Pourquoi `timeout=5` dans les requêtes ? »
- « Que se passe-t-il si l'API n'est pas lancée ? (tester le cas) »

V6 Tests — Partie 2

03_test.py — Cas limite texte vide (TODO 3)

🕒 12h00 – 12h15 · 15 min

🎯 TON RÔLE

Tu completes le test du cas limite (texte vide) et tu animes une session de tests libres : la classe propose des phrases inattendues pour trouver les limites du modèle.

📝 TODO À COMPLÉTER

#	Fonction à compléter	Ce qu'il faut faire	✓
3	<code>tester_texte_vide()</code>	<code>requests.post('/predict', json={'texte': ''})</code> → vérifier <code>status_code == 400</code>	☐

► SESSION DE TESTS LIBRES — INVITER LA CLASSE

```
python 03_test.py # tous les tests doivent passer (7/7)
```

Textes à tester (classe)	Résultat attendu / discussion
"Ce produit est correct, sans plus."	Ambigu → score faible
"Bien."	Très court → score peu fiable
"This product is great!"	Anglais → hors distribution
"Yamba, ne y yemba!"	Mooré → résultat imprévisible
"(texte vide)"	→ erreur 400 (déjà testé)

? QUESTIONS À POSER À LA CLASSE

- « Un bon système devrait-il accepter les textes en mooré ? Comment l'améliorer ? »
- « Comment tester automatiquement les cas limites dans une CI/CD ? »
- « Que retourne l'API pour 'Bien.' ? Est-ce fiable ? Pourquoi ? »

V7

Débrief & Synthèse

Consolider les apprentissages et ouvrir vers la suite

🕒 12h15 – 13h00 · 45 min

🎯 TON RÔLE

Tu animes le débrief collectif. Tu n'as pas de code à exécuter — ton rôle est de faire parler la classe, de consolider le pattern de déploiement et d'ouvrir vers la Phase 2.

📋 DÉROULÉ SUGGÉRÉ (45 MIN)

- Tour de table rapide — 1 min par groupe (10 min) :
 - « Ce que votre groupe a réussi »
 - « Le TODO qui vous a le plus bloqué »
- Synthèse du pattern de déploiement — au tableau (10 min)
- Questions de réflexion (15 min)
- Ouverture Phase 2 (10 min)

🔄 LE PATTERN À CONSOLIDER AU TABLEAU



Training → Sérialisation (.pkl) → API REST (/predict) → Interface web (/app) → DockerCe même enchaînement s'applique à CamemBERT, à Qwen3-8B, à YOLO. C'est la compétence de déploiement.

❓ QUESTIONS DE RÉFLEXION À LANCER AU GROUPE

- « Pourquoi utilise-t-on pickle ? Quelles sont ses limites de sécurité ? »
- « Que se passe-t-il si on déploie le vectorizer.pkl sur une machine avec une version différente de sklearn ? »
- « Comment passer du mode mock (TF-IDF) au vrai CamemBERT ? Que faut-il changer dans 02_serve.py ? »
- « Comment mesurer la latence de votre API sous charge (100 requêtes simultanées) ? »
- « Votre modèle est en production. Un avis en mooré arrive. Que faites-vous ? »

🚀 OUVERTURE — PHASE 2 (HACKATHON)

Le même pattern que vous venez de maîtriser s'applique à un LLM. Les différences sont de degré, pas de nature :

	Phase 1 (aujourd'hui)	Phase 2 (hackathon)
Modèle	TF-IDF + LogReg (~1 Mo)	Qwen3-8B-Q4 (5 Go)
Sérialisation	.pkl (pickle)	GGUF (llama.cpp)
Serving	uvicorn directement	llama-server (process séparé)
Endpoint	POST /predict	POST /ask
Latence	< 50 ms	5 – 30 s
Cas d'usage	Sentiment	RAG sur arrêtés MESRSI